

From Variability Knowledge to Sovereign Multi-Cloud Deployment: A Low-Code DSL for Adaptive MLOps Pipelines

An industrial-partner contribution to the ANR AdaptiveMLOps project

Xavier Quesnot
GetCaaS, France
xavier@getcaas.io

Draft v0.1 — September 25, 2026 — for consortium review

Abstract

MLOps pipelines must be continuously retrained to cope with data and concept drift, and increasingly operated across several cloud providers under cost, carbon and sovereignty constraints. Designing them requires knowledge spread across data scientists, ML engineers and operations engineers. Building on the variability mapping and the SkeltyMLOps reference architecture produced within the ANR AdaptiveMLOps project, we present *amlops*, an open-source prototype that (i) formalises MLOps variability as a feature model whose six top-level categories come from a prior mapping study, complemented by documented best-practice rules; (ii) offers a two-level YAML domain-specific language in which a non-expert writes a profile and objectives while an expert refines features, steps and parameters; (iii) derives a platform-independent pipeline model following a software-process-line view, including executable co-ordination contracts; (iv) places pipeline steps across providers with an exact multi-objective optimiser and compiles the result to Terraform, Argo Workflows and CI artefacts; and (v) re-replaces services and schedules drift-triggered re-training at run time. We evaluate the prototype on four illustrative case studies with an illustrative provider catalogue and synthetic environment traces. Generated artefacts realise 21/31 of the non-planning SkeltyMLOps activities; 34 of 44 testable leaf features have an observable effect on the output, the others pinpointing missing generators. Static carbon-aware placement cuts operational carbon by 91–92% versus a cost-only placement for 5.6–11.1% extra cost. In contrast, hourly re-placement of serving components brings little: even a clairvoyant oracle improves the normalised objective by at most $\approx 2\%$ over a static placement, while a naive reactive policy migrates ≈ 102 times a month and worsens it by 27–30%. We discuss what these results mean for the design of adaptive MLOps tooling and what real industrial traces must now confirm.

1 Introduction

MLOps extends DevOps to the development and operation of ML-powered software [1, 2]. Once deployed, models face changing data distributions and degrade silently [3]; pipelines therefore embed monitoring, retraining triggers and redeployment. In practice these pipelines are assembled from heterogeneous tools and platforms, and must satisfy non-functional and organisational requirements: cost, latency, environmental footprint, and, in Europe, data residency and qualified hosting (e.g. SecNumCloud or HDS in France).

The ANR AdaptiveMLOps project (ANR-24-IAS2-0004) studies how domain engineering (feature models, software product and process lines) and model-driven engineering (MDE) can capture this variability and assist the design of new pipelines. Two first results of the consortium frame this paper: a variability mapping study that identified six variability categories and eight modelling requirements R1–R8 [4], and SkeltyMLOps, a reference architecture built on 38 consolidated activities and on coordination contracts executed by a process orchestrator [5, 6].

This paper reports the industrial partner’s contribution: a tool chain that turns this knowledge into deployable, re-deployable artefacts on a sovereign multi-cloud offer. It addresses three research questions:

- RQ1** Can the consortium’s variability and reference-architecture knowledge drive a low-code DSL usable at two levels of expertise?
- RQ2** Can a single DSL model be pivotal, i.e. compiled into Infrastructure- and Platform-as-Code for several providers with traceability?
- RQ3** What do static and dynamic multi-provider placement bring in cost and carbon, and at which operational risk?

Contributions: (1) an executable feature model of MLOps variability with best-practice rules and automatic, re-validated fixes; (2) a two-level DSL and a derivation engine based on a process-line base model with variation points;

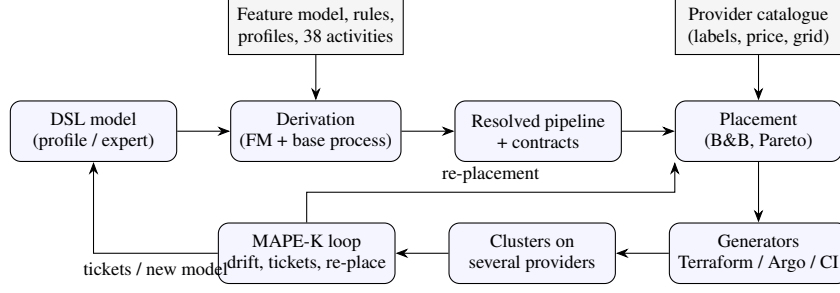


Figure 1: The amlops tool chain.

(3) an operational semantics for SkeltyMLOps coordination contracts; (4) an exact multi-objective placement optimiser and model-to-text generators; (5) a simulation study of re-deployment policies whose negative result on hourly re-placement is, we argue, informative. All code, examples and experiment scripts are released under Apache-2.0; every number in this paper is generated by the experiment script (Sect. 10).

2 Background and related work

Feature models and process lines. Feature models (FODA [7]) describe the common and variable features of a family; their semantics maps to propositional logic, enabling automated analysis [8, 9]. Software process lines apply product-line ideas to processes, under the rationale that software processes are software too [10]; BPMN* [11] annotates process models with variation points. The consortium’s mapping study applied this view to MLOps and derived eight requirements: control flow and conditional execution (R1), optional and alternative steps (R2), component composition (R3), reactive orchestration (R4), multi-dimensional variability (R5), domain-specific extensions (R6), constraints and dependencies (R7) and configuration resolution (R8) [4].

MLOps architectures and MDE. Kreuzberger et al. [1] and Najafabadi et al. [12] synthesise MLOps components; SkeltyMLOps [5, 6] organises 38 activities into five dimensions (Plan, Data, Model, Soft, Ops) around a process orchestrator that executes coordination contracts made of tickets, events, gates and hand-offs. DevOpsML [13] weaves process and platform models for DevOps but, as noted by [4], without variability support. Infrastructure as Code has been widely studied [14], but IaC is usually written per provider rather than derived from a higher-level pipeline model.

Carbon-aware computing. Temporal shifting of deferrable workloads towards low-carbon hours [15] and spatio-temporal load shaping in data centres [16] show that flexibility can reduce emissions. We study how much of this potential is reachable for MLOps pipelines under sovereignty and latency constraints, and at which migration cost.

3 Approach overview

Fig. 1 summarises the tool chain. A *knowledge layer* packages the feature model, rules, profiles, the SkeltyMLOps activity vocabulary and a provider catalogue. A user model (DSL) is *derived* into a resolved, platform-independent pipeline; the *placement* optimiser binds each step to an offer (provider, region, instance type); *generators* emit IaC/PaC; at run time a MAPE-K loop [17] detects drift, opens tickets under the contracts and recommends re-placements.

4 Knowledge layer

Feature model. The reference model has 69 features (50 leaves) and 10 cross-tree constraints. Its six top-level subtrees are the categories of [4]: *Functional* (ML task, pre-processing, tuning, explainability), *Behavioral* (training triggers, deployment strategy, human gate), *Technical* (orchestrator, serving, tracking, data versioning, runtime, GPU), *Domain-specific*, *Non-functional* (objectives, monitoring signals) and *Organizational* (residency, SecNumCloud, compliance frames). The constraint reported in the consortium’s poster, $\neg(\text{ClassificationBaseline} \wedge \text{DimReduction})$, is included verbatim; others encode semantic dependencies, e.g. a drift trigger requires a drift signal, a high-risk system under the AI Act requires a human gate and explainability, HDS hosting implies a residency constraint. The leaf features are an initial proposal by the industrial partner, to be validated by the consortium.

Configuration completion (R8) is a depth-first search over undecided features in pre-order with Kleene three-valued evaluation of all structural rules and constraints for pruning; it returns a minimal completion (false-first except for features flagged as defaults) or reports the violated rules.

Best-practice rules. Know-how that should not be a hard constraint is stored as 8 advisory rules, each with a condition, a rationale, a source and, when possible, an automatic fix expressed as a feature delta. A fix is proposed only if the resulting configuration is valid. For instance, BP01 fires when retraining is drift-triggered without data versioning, which would make automatic retraining non-reproducible.

Profiles. 4 profiles are named partial configurations (e.g. *regulated-health-classifier*) that act as low-code entry points.

5 DSL and derivation

The concrete syntax is YAML. A minimal model and an expert refinement are shown in Listing 1. The abstract syntax distinguishes the user model from the resolved pipeline (steps typed by SkeltyMLOps activities, triggers, contracts, objectives, placement constraints).

Listing 1: Non-expert model (top) and excerpt of an expert model (bottom).

```
pipeline: churn-prediction
profile: tabular-classification-continuous
data: {source: "s3://datalake/crm/churn.parquet", volume_gb: 40}
objectives: {cost: 0.6, carbon: 0.4}
-----
profile: anomaly-detection-edge-iot
best_practices: true # apply the advisor's automatic fixes
features: {select: [CarbonAware, GPU, HyperparameterTuning, EURidency]}
placement: {allowed_providers: [scaleway, ovhcloud, outscale], colocate: [[serve, monitor]]}
steps:
  - {id: window_features, kind: transform, activity: D5, after: remove_outliers}
  - {id: train, hours: 3, params: {algorithm: isolation_forest}}
triggers: [{type: drift, detector: ks, threshold: 0.1}]
overrides: {steps.serve.replicas: 3}
```

Derivation (i) completes the configuration from the profile and user selections; (ii) optionally applies valid best-practice fixes; (iii) instantiates a base process whose variation points are bound by features: *Preprocess* expands into one step per selected pre-processing feature, *Deploy* depends on the deployment strategy (a test deployment is inserted for canary, shadow and blue-green), *Serve* is online or batch, *Monitor* carries the selected signals; optional steps (tuning, explanation, human approval) follow optional features; (iv) merges user steps by identifier, inserting new ones in the chain; (v) applies dotted-path overrides; (vi) derives triggers and coordination contracts from behavioural features and placement labels from organisational features. Every produced element records a trace link to its cause. New step kinds and generators are registered through Python entry points, which covers domain-specific extensions (R6) without changing the DSL.

Executable coordination contracts. SkeltyMLOps describes contracts in prose [6]. We give them an operational semantics: a ticket opened under a contract advances gate by gate only when the gate's owner submits the required artefacts and, if requested, an explicit approval; conditional gates (e.g. `model.input_schema != api.current_schema` for the API-impact gate) are evaluated on the ticket context and skipped when false; every transition is appended to an auditable trace. The derived drift-response contract reproduces the walkthrough of [6]; the human gates are active only when the *HumanGate* feature is selected, which the AI Act constraint enforces.

6 Placement and generation

Static placement. Each step s has resource needs and a monthly usage; each offer o has a price, a power draw, a PUE, a grid carbon intensity, a latency and labels. With x_s the chosen offer, we minimise

$$J = w_c \frac{\sum_s \text{cost}(s, x_s) + \sum_{(a,b)} \text{egress}(a, x_a, x_b)}{C_{\text{ref}}} + w_g \frac{\sum_s \text{kWh}(s, x_s) \cdot \text{grid}(x_s)}{G_{\text{ref}}} + w_l \frac{\sum_{s \in \text{serve}} \text{lat}(x_s)}{L_{\text{ref}}}$$

subject to resource fit, required labels, allowed providers, pins, co-location groups and a latency bound. The references are sums of per-step minima, so $J \geq 1$ when $\sum w = 1$, and $J = 1$ means every step is at its individual optimum. We solve it exactly by depth-first branch and bound in topological order, with the admissible bound “partial objective + sum of per-step minima” (egress is non-negative) and a location-level dominance reduction (per step only the best offer of each location can be optimal). Optimality was checked against brute force in the test suite. A weight sweep yields the cost-carbon Pareto front.

Table 1: Leaf features per category: total, consistently flippable, and with an observable effect on the output.

Category	Leaves	Tested	Effective
Functional	12	8	7
Behavioral	10	10	10
Technical	11	9	2
DomainSpecific	5	5	3
NonFunctional	6	6	6
Organizational	6	6	6

Generation. Terraform is emitted with native Scaleway resources (Kapsule cluster, private network, node pools, versioned bucket) and module stubs for other providers. For each location a Kubernetes cluster hosts an Argo WorkflowTemplate with the batch steps placed there; the human gate becomes an Argo suspend node; cross-location hand-offs are Argo Events webhooks and sensors; schedule triggers become CronWorkflows and drift triggers sensors fed by the monitor; serving and monitoring become Deployments (or a KServe InferenceService) with ConfigMaps holding the drift policy and the contracts. A GitHub Actions workflow re-generates the artefacts, fails on diff (the model is the source of truth), validates Terraform and schema-checks manifests. A `trace.json` maps each file to the model elements and activities it realises.

Run-time adaptation. A MAPE-K loop monitors production batches with PSI and two-sample Kolmogorov–Smirnov detectors, opens a ticket under the matching contract, and compares the current placement with a re-optimised one under new conditions. For the dynamic study we consider the serving group (serve and monitor, co-located) and drift-triggered retraining jobs, and four policies: *static* (placed on mean conditions, moves only on outage and then returns), *reactive* (follows the hourly argmin, retrains immediately at the best location), *hysteresis* (migrates only if the forecast gain over a look-ahead window exceeds the migration cost times $1 + m$; retraining jobs are shifted within their deferrable window to the lowest forecast objective), and a clairvoyant *oracle* (dynamic programming over locations with switching costs, true future values for jobs), which is a lower bound rather than a policy. Forecasts are seasonal-naive (value 24 h earlier).

7 Evaluation

Setup. Four illustrative case studies: churn prediction (non-expert, 5 DSL lines), clinical triage support (regulated profile, 5 lines), predictive maintenance on industrial IoT (expert, 28 lines) and retail demand forecasting (expert, batch, 16 lines). The provider catalogue contains 27 offers in 9 locations of four provider archetypes (two French clouds, a SecNumCloud/HDS-qualified provider, a hyperscaler). *All prices, power draws and grid intensities are order-of-magnitude placeholders, not quotes*; environment traces (hourly grid intensity with diurnal profiles, solar dips and AR(1) noise, random outages) are synthetic and seeded. Results characterise the method, not providers.

RQ1 — Knowledge coverage. We flip each leaf feature in the context of a reference profile and check whether the derived model or the generated artefacts change (Table 1). 44 of 50 leaves can be flipped consistently; 34 have an observable effect. Behavioural, non-functional and organisational features are all effective. The inert ones are Airflow, ArgoWorkflows, DataVersioning, ExperimentTracking, Industry40IoT, Kubeflow, Kubernetes, Retail, Serverless, TabularClassification: they reveal that the current generators target only Argo on Kubernetes and emit nothing for data versioning or experiment tracking, and that the ML task and some domains do not yet change the pipeline. This metric turns the feature model into a test oracle for generator completeness.

Regarding the reference architecture, generated artefacts realise 21/38 SkeltyMLOps activities: Ops 9/9, Data 5/8, Model 5/9, Soft 2/5, Plan 0/7. Plan activities (conceptualising, requirements, architecture design) are human by nature and are supported by the DSL itself rather than by generated code; the uncovered Data, Model and Soft activities (collecting and labelling data, experimenting, developing model code, building and packaging) are the developer-side work that a deployment DSL deliberately leaves to users.

RQ2 — Pivotal model. Table 2 relates model size to generated artefacts. A 5-line non-expert model yields 10–11 steps and about 400 lines of Terraform, Argo and CI configuration; expert models are larger (they carry parameters) and the ratio drops to 18.3. Line ratios are only a proxy for effort; they do not measure correctness of the generated code, which we check by parsing all manifests, asserting DNS-compatible Argo names and native Terraform resources in the test suite, but not yet by deployment on real clusters.

Table 2: Model size vs. generated artefacts (non-comment lines). Findings: fired best-practice rules.

Case	DSL lines	Steps	Files	Generated lines	Ratio	Findings
churn	5	10	7	396	79.2	2
health	5	11	7	403	80.6	3
maintenance	28	14	7	513	18.3	1
retail	16	10	5	364	22.8	0

Table 3: Static placement. Cost in EUR/month, carbon in kgCO₂e/month, J normalised objective (1 = per-step optimum).

Case	Providers	Cost	Carbon	J	J best single	J worst single	Cost (cost-only)	Carbon (cost-only)	B&B nodes
churn	4	210	14.5	1.086	1.086	1.470	199	168.1	91
health	1	332	15.0	1.000	1.000	1.000	332	15.0	12
maintenance	3	305	18.9	1.027	1.027	1.313	290	25.6	343403
retail	4	77	4.8	1.094	1.094	1.322	70	58.8	91

RQ3a — Static placement. Table 3 compares the optimum with single-provider optima and with a cost-only placement. Three observations. (1) The multi-provider optimum equals the best single-provider optimum (to three decimals) in all cases: the value lies in choosing the right provider and region per pipeline, not in splitting a pipeline, because egress and co-location penalise splits. (2) Choosing badly is expensive: the worst feasible single provider has an objective up to 35% above the optimum (churn: 1.470 vs. 1.086). (3) Carbon-aware weights cut operational carbon by 91% (churn) and 92% (retail) versus the cost-only placement for +5.6% and +11.1% cost; this gap is driven by the catalogue’s cheaper, high-carbon region and must be re-measured with real prices. For the regulated case, the HDS label leaves a single feasible provider. The exact solver needs 91–271 nodes on chains of 10–30 steps; ties between near-equivalent French regions raise this to 3.4×10^5 nodes (a few seconds) for the maintenance case.

RQ3b — Dynamic re-deployment. We simulate 30 days (hourly) for the maintenance pipeline with equal cost and carbon weights, 20 seeds, in two scenarios: A, all EU locations allowed; B, French locations excluded (e.g. a residency constraint towards other EU countries), leaving two serving locations whose grids differ in daily shape. Table 4 reports means over seeds; J is normalised by the static policy of the same seed.

The upside of hourly re-placement is small: the clairvoyant oracle improves J by only 1.7% (A, $J = 0.983$) and 0.7% (B, $J = 0.993$) with 2–3 migrations a month. The reactive policy migrates 102 (A) and 90 (B) times a month; it saves 8.8% and 3.9% of carbon but raises cost by 69% and 58%, hence $J = 1.301$ and 1.271. Hysteresis avoids this churn (< 1 migration a month) but does not capture the oracle’s gain: $J = 1.000$ and 1.009. The ablation (Table 5) shows the same picture for margins 0–1 and look-aheads of 3–12 h. A diagnosed weakness is that, after a forced move caused by an outage, hysteresis only returns if the short-horizon forecast gain covers the migration cost, which the static policy does unconditionally; this explains its higher carbon variance in scenario B. Temporal shifting of retraining jobs is where flexibility pays in relative terms: job carbon drops by 11% (A) with a 12 h window, and the oracle shows that better forecasts could reach about 20%; retraining is however a small share of the total here.

8 Discussion

For tool design. Under these assumptions, the placement decision that matters is the initial, constraint- and objective-aware choice of provider and region; it should be made explicit and traceable at design time, which the DSL does. Dynamic re-placement of long-running services should be conservative by default, triggered by events (outages, price or catalogue changes, drift-induced retraining) rather than by hourly signals; temporal shifting of deferrable training is the cheap lever. These are statements about the method on synthetic inputs; the magnitudes will change with real traces, larger state to migrate, or regions with more volatile grids.

Relation to the consortium’s work. The prototype reuses the six variability categories, the requirements R1–R8 and the SkeltyMLOps activities and contracts. It addresses R2, R5, R7, R8 through the feature model and derivation, R1 and R4 through triggers, conditional gates and Argo DAGs, R3 and R6 through step-kind and generator registries. It does not yet offer a BPMN or graphical process view, nor a metamodel in a standard MDE technical space (e.g. Ecore); both are natural bridges to the academic partners’ work.

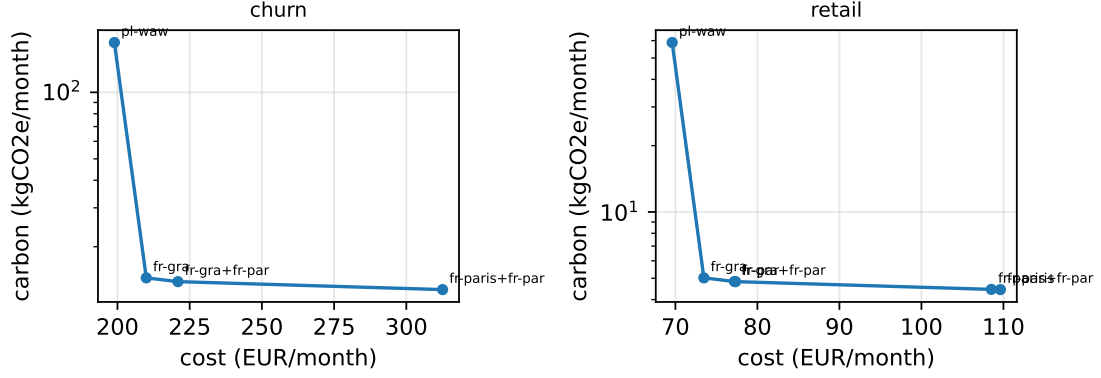


Figure 2: Cost-carbon Pareto fronts (weight sweep); labels are the regions used.

Table 4: Dynamic policies over 30 days (mean $\pm$ sd over 20 seeds). Job carbon: drift-triggered retraining only.

Scen.	Policy	Cost (EUR)	Carbon (kg)	Migrations	J	Job carbon (kg)
A	static	308 $\pm$ 15	19.3 $\pm$ 0.7	0.9	1.000	0.91
A	reactive	519 $\pm$ 30	17.6 $\pm$ 0.5	101.8	1.301	0.85
A	hysteresis	306 $\pm$ 15	19.4 $\pm$ 0.6	0.4	1.000	0.82
A	oracle	310 $\pm$ 14	18.5 $\pm$ 0.6	3.1	0.983	0.72
B	static	315 $\pm$ 12	109.3 $\pm$ 3.4	0.8	1.000	5.13
B	reactive	497 $\pm$ 27	105.0 $\pm$ 3.4	90.1	1.271	5.01
B	hysteresis	312 $\pm$ 11	112.2 $\pm$ 6.8	0.5	1.009	5.02
B	oracle	315 $\pm$ 11	107.6 $\pm$ 3.5	1.9	0.993	4.37

9 Threats to validity

Construct: J aggregates heterogeneous criteria through weights and per-step references; lines of generated code are a weak proxy for effort; carbon is operational and location-based only (no embodied emissions, no network energy). *Internal:* the leaf features, rules and constraints are the author’s proposal and have not yet been validated by the consortium; migration cost is a fixed penalty plus egress. *External:* prices, power draws and grid intensities are placeholders and the environment is synthetic; case studies are illustrative, not the industrial cases announced in the project. *Conclusion:* 20 seeds give tight intervals on synthetic data but say nothing about real variability. Generated artefacts are parsed and unit-tested, not yet deployed.

10 Conclusion and roadmap

We presented amlops, a variability-aware low-code DSL that compiles MLOps pipelines to multi-provider IaC/PaC and adapts them at run time, grounded in the variability and reference-architecture results of the AdaptiveMLOps consortium. Next steps are: (1) consortium review of the feature model and rules; (2) replacement of the catalogue and traces by contractual prices and measured grid data, and deployment of generated artefacts on the industrial partner’s platform; (3) generators for experiment tracking, data versioning and alternative orchestrators, guided by the inert-feature metric; (4) a return-to-preferred-location rule and event-driven triggers for re-placement; (5) an Ecore metamodel and a BPMN view aligned with the process-line modelling of the academic partners; (6) evaluation with non-expert users.

Data and code availability

Source code, examples, tests and the experiment script (experiments/run_all.py) are provided in the accompanying repository under Apache-2.0. All tables, figures and numbers in this paper are produced by that script (paper/generated/).

Table 5: Hysteresis ablation, scenario B (10 seeds).

Margin m	Look-ahead (h)	J	Migrations
0.0	3	1.009	0.3
0.0	6	1.003	0.6
0.0	12	1.004	1.1
0.25	3	1.009	0.3
0.25	6	1.009	0.4
0.25	12	1.004	1.0
1.0	3	1.009	0.3
1.0	6	1.009	0.3
1.0	12	1.009	0.4

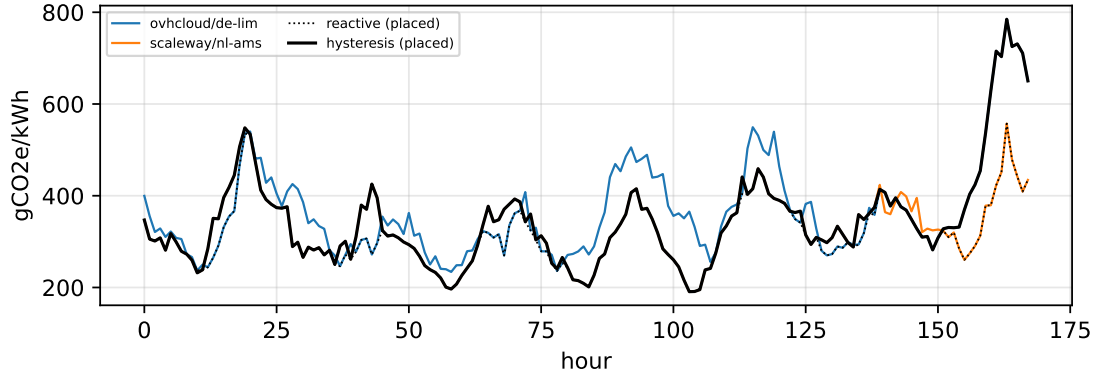


Figure 3: Scenario B, first week, seed 0: grid intensity of the two feasible serving locations and intensity experienced by the placed service.

Acknowledgements

This work is part of the AdaptiveMLOps project funded by the French National Research Agency (ANR-24-IAS2-0004).

Declaration on generative AI

A large language model assistant was used to help write code and draft this text; the author reviewed the design, the code, the experiments and the manuscript and takes responsibility for the content.

References

- [1] D. Kreuzberger, N. Kühl, S. Hirschl. Machine Learning Operations (MLOps): Overview, definition, and architecture. *IEEE Access* 11:31866–31879, 2023.
- [2] D. Sculley et al. Hidden technical debt in machine learning systems. *NeurIPS*, 2015.
- [3] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, A. Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys* 46(4), 2014.
- [4] C. El Hatimi, V. Berry, A.-D. Seriai, V. Preney, X. Quesnot, C. Tibermacine, C. Urtado, S. Vauttier. Toward Adaptive MLOps: Variability mapping and modeling. *Journées nationales du GdR GPL*, Pau, 2025. hal-05127859.
- [5] C. Daoud, D. Azar, J. Boiché, C. Urtado, S. Vauttier. SkeltyMLOps: Orchestrating collaborative MLOps activities. *MLOps25 Workshop, ECAI 2025*, pp. 55–66. hal-05337700.
- [6] C. Daoud, D. Azar, C. Urtado, S. Vauttier. A reference architecture for an orchestrated collaborative MLOps: A ground-up design from literature combining human and LLM expertise. *Future Generation Computer Systems* 185:108700, 2026.
- [7] K. C. Kang, S. G. Cohen, J. A. Hess, W. E. Novak, A. S. Peterson. Feature-Oriented Domain Analysis (FODA) feasibility study. CMU/SEI-90-TR-21, 1990.

- [8] D. Batory. Feature models, grammars, and propositional formulas. *SPLC*, 2005.
- [9] D. Benavides, S. Segura, A. Ruiz-Cortés. Automated analysis of feature models 20 years later: A literature review. *Information Systems* 35(6), 2010.
- [10] L. Osterweil. Software processes are software too. *ICSE*, 1987.
- [11] M. Terenciani, D. M. B. Paiva, G. B. Landre, M. I. Cagnin. BPMN* – A notation for representation of variability in business process towards supporting business process line modeling. *SEKE*, 2015.
- [12] F. Amou Najafabadi, J. Bogner, I. Gerostathopoulos, P. Lago. An analysis of MLOps architectures: A systematic mapping study. *ECSA*, 2024.
- [13] A. Colantoni, L. Berardinelli, M. Wimmer. DevOpsML: Towards modeling DevOps processes and platforms. *MODELS Companion*, 2020.
- [14] A. Rahman, R. Mahdavi-Hezaveh, L. Williams. A systematic mapping study of infrastructure as code research. *Information and Software Technology* 108, 2019.
- [15] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, L. Thamsen. Let’s wait awhile: How temporal workload shifting can reduce carbon emissions in the cloud. *Middleware*, 2021.
- [16] A. Radovanović et al. Carbon-aware computing for datacenters. *IEEE Transactions on Power Systems* 38(2), 2023.
- [17] J. O. Kephart, D. M. Chess. The vision of autonomic computing. *Computer* 36(1), 2003.